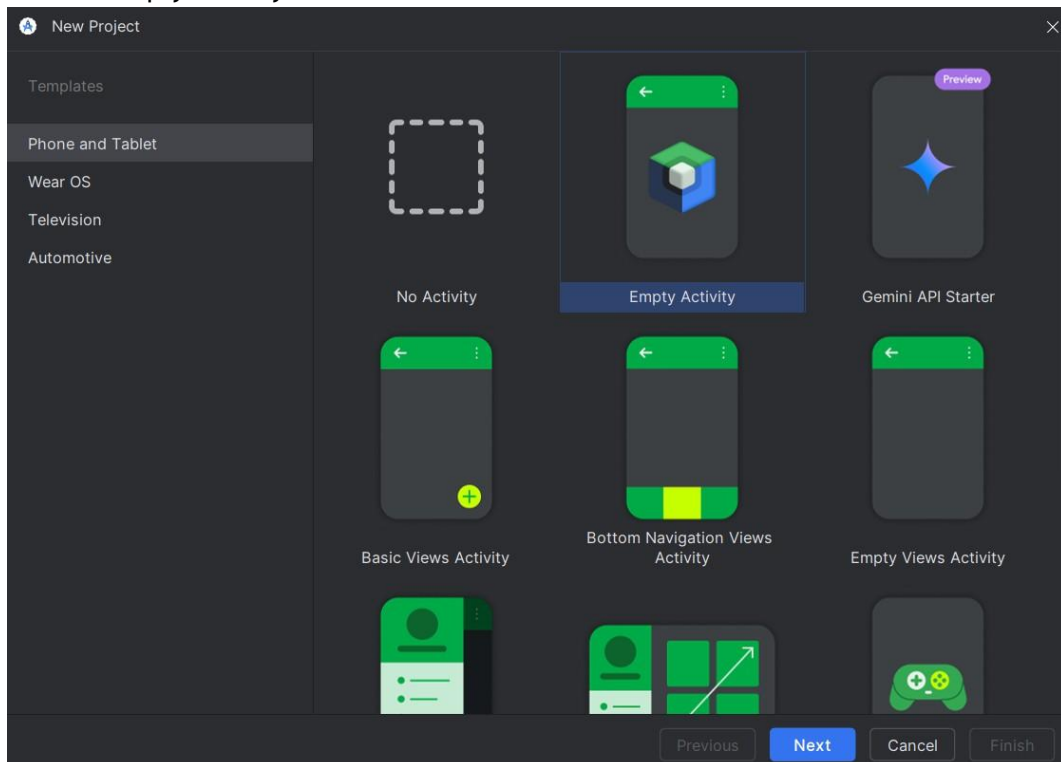
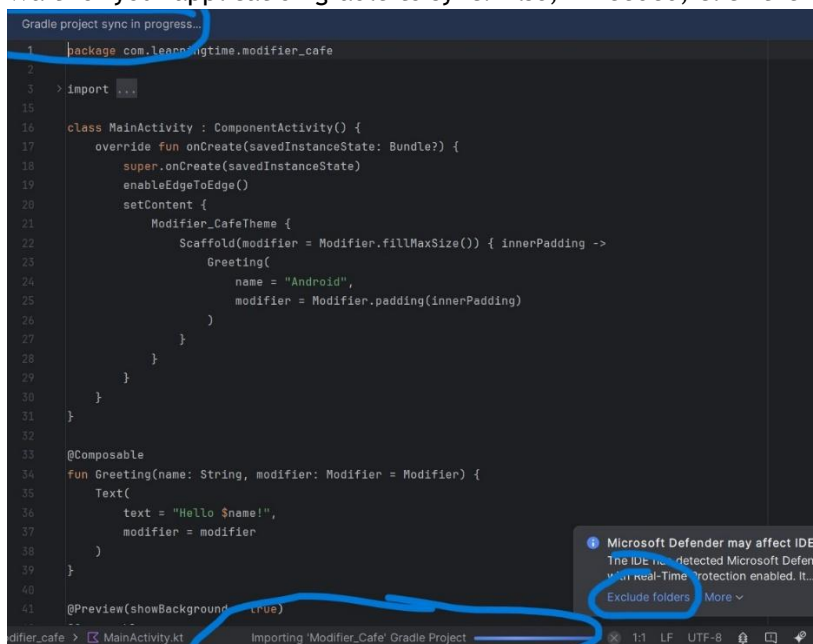


1. Open Android Studio and Create a new Project
2. Click on Empty Activity



3. Write a name for your Application and then click Finish.
4. Wait for your application gradle to sync. Also, if needed, click the exclude folders option.



5. After the sync, your application will be available to run.

Introduction to MVVM Architecture and Kotlin Data Classes

MVVM stands for:

Model

View

ViewModel

MVVM is a way of organising an Android application.

Instead of writing all the code inside one file, the application is divided into separate parts.

Model → Stores data

View → Displays the user interface

ViewModel → Manages the data and application logic

In our counter application:

Model

The Model is a data class that stores the counter value.

```
data class CounterState(  
    val count: Int = 0  
)
```

View

The View is the Jetpack Compose screen.

It displays:

- The counter value
- The Increment button
- The Decrement button
- The Reset button

ViewModel

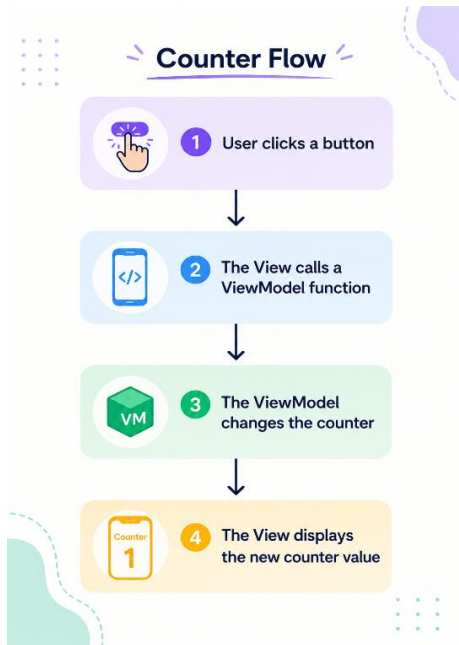
The ViewModel:

- Stores the current counter state
- Increases the counter
- Decreases the counter

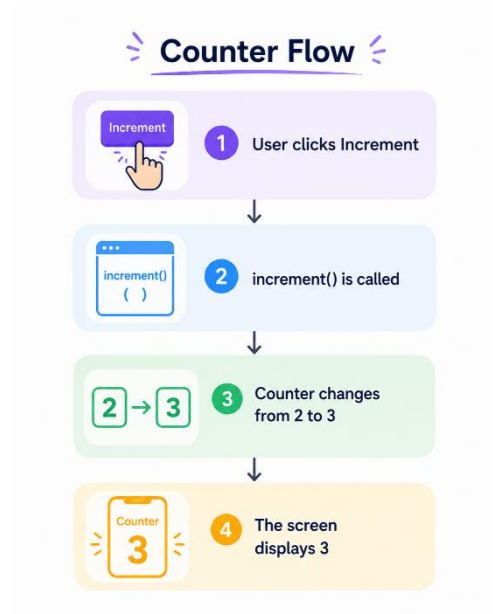
- Resets the counter

Application Flow

The application works like this:



Example:



Create the Project

Create a new Android Studio project.

Use:

Template: Empty Activity

Language: Kotlin

User Interface: Jetpack Compose

Suggested project name:

SimpleMVVMCounter

Project Files

Create the following files:

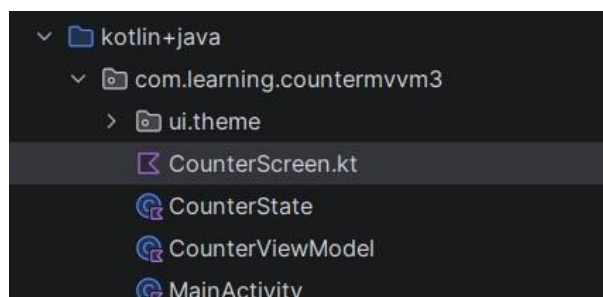
MainActivity.kt

CounterState.kt

CounterViewModel.kt

CounterScreen.kt

For Example:



Step 1: Create the Data Class

Create a Kotlin file named:

`CounterState.kt`

Add the following code:

```
package com.learning.countermvvm3
```

```
data class CounterState(  
    val count: Int = 0  
)
```

Understanding the Data Class

A data class is used to store data.

Our data class stores one value:

```
val count: Int
```

The default counter value is:

```
0
```

Therefore, when we create:

```
CounterState()
```

the value will be:

```
count = 0
```

Why Use a Data Class?

A data class helps us group related values together.

For example, later we could add more information:

```
data class CounterState(  
    val count: Int = 0,  
    val message: String = "Start counting"  
)
```

For this simple application, we will use only the counter value.

The copy() Function

Kotlin data classes automatically provide a function called:

`copy()`

It creates a new object by changing one or more values.

Example:

```
val oldState = CounterState(  
    count = 5  
)  
  
val newState = oldState.copy(  
    count = 6  
)
```

Now:

`oldState count = 5`

`newState count = 6`

We will use `copy()` inside the `ViewModel`.

Step 2: Create the ViewModel

Create a Kotlin file named:

`CounterViewModel.kt`

Add the following code:

```
package com.learning.countermvvm3  
  
import androidx.compose.runtime.State  
import androidx.compose.runtime.mutableStateOf  
import androidx.lifecycle.ViewModel  
  
class CounterViewModel : ViewModel() {  
  
    private val _counterState = mutableStateOf(
```

```

        CounterState()
    )

    val counterState: State<CounterState> = _counterState

    fun increment() {
        _counterState.value = _counterState.value.copy(
            count = _counterState.value.count + 1
        )
    }

    fun decrement() {
        if (_counterState.value.count > 0) {
            _counterState.value = _counterState.value.copy(
                count = _counterState.value.count - 1
            )
        }
    }

    fun reset() {
        _counterState.value = CounterState()
    }
}

```

Understanding the ViewModel

The ViewModel class is:

```
class CounterViewModel : ViewModel()
```

It inherits from Android's ViewModel class.

The ViewModel stores and manages the screen data.

Creating the Counter State

The following code creates the initial state:

```
private val _counterState = mutableStateOf(  
    CounterState()  
)
```

CounterState() creates:

```
count = 0
```

mutableStateOf() makes the value observable by Jetpack Compose.

When the value changes, Compose updates the screen.

Increment Function

The increment function is:

```
fun increment() {  
    _counterState.value = _counterState.value.copy(  
        count = _counterState.value.count + 1  
    )  
}
```

Suppose the current count is:

```
4
```

The function calculates:

```
4 + 1 = 5
```

The new state becomes:

```
count = 5
```

Decrement Function

The decrement function is:

```
fun decrement() {  
    if (_counterState.value.count > 0) {  
        _counterState.value = _counterState.value.copy(  
            count = _counterState.value.count - 1  
        )  
    }  
}
```

```
        )  
    }  
}
```

The condition:

```
if (_counterState.value.count > 0)
```

prevents the counter from becoming negative.

For example:

Current count = 0

User clicks Decrement

Result = 0

Reset Function

The reset function is:

```
fun reset() {  
    _counterState.value = CounterState()  
}
```

CounterState() creates the default value:

count = 0

Therefore, the counter returns to zero.

Step 3: Create the View

Create a Kotlin file named:

CounterScreen.kt

Add the following code:

```
package com.learning.countermvvm3  
  
import androidx.compose.foundation.layout.Arrangement  
import androidx.compose.foundation.layout.Column  
import androidx.compose.foundation.layout.PaddingValues  
import androidx.compose.foundation.layout.Row  
import androidx.compose.foundation.layout.Spacer  
import androidx.compose.foundation.layout.height  
import androidx.compose.foundation.layout.padding  
import androidx.compose.material3.Button
```

```

import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.lifecycle.viewmodel.compose.viewModel

@Composable
fun CounterScreen(
    modifier: Modifier = Modifier,
    contentPadding: PaddingValues = PaddingValues(0.dp),
    counterViewModel: CounterViewModel = viewModel()
) {
    val state by counterViewModel.counterState

    Column(
        modifier = modifier
            .padding(contentPadding)
            .padding(24.dp),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
            text = "MVVM Counter",
            fontSize = 28.sp
        )

        Spacer(
            modifier = Modifier.height(30.dp)
        )

        Text(
            text = state.count.toString(),
            fontSize = 60.sp
        )

        Spacer(
            modifier = Modifier.height(30.dp)
        )

        Row(
            horizontalArrangement = Arrangement.spacedBy(16.dp)
        ) {
            Button(
                onClick = {
                    counterViewModel.decrement()
                }
            ) {
                Text("Decrease")
            }
        }
    }
}

```

```

        Button(
            onClick = {
                counterViewModel.increment()
            }
        ) {
            Text("Increase")
        }
    }

    Spacer(
        modifier = Modifier.height(16.dp)
    )

    Button(
        onClick = {
            counterViewModel.reset()
        }
    ) {
        Text("Reset")
    }
}

```

Understanding the View

The View obtains the ViewModel using:

```
counterViewModel: CounterViewModel = viewModel()
```

The `viewModel()` function creates or retrieves the ViewModel.

Do not create the ViewModel manually like this:

```
val counterViewModel = CounterViewModel()
```

Use:

```
viewModel()
```

Reading the State

The following line reads the state from the ViewModel:

```
val state by counterViewModel.counterState
```

The counter value can now be accessed using:

```
state.count
```

The counter is displayed using:

```
Text(  
    text = state.count.toString()  
)
```

When the counter changes, Compose automatically updates this Text.

Calling ViewModel Functions

The Increase button calls:

```
counterViewModel.increment()
```

The Decrease button calls:

```
counterViewModel.decrement()
```

The Reset button calls:

```
counterViewModel.reset()
```

The buttons do not directly change the value.

They ask the ViewModel to change it.

Step 4: Update MainActivity

Open:

`MainActivity.kt`

Use the following code:

```
package com.learning.countermvvm3  
  
import android.os.Bundle  
import androidx.activity.ComponentActivity  
import androidx.activity.compose.setContent  
import androidx.activity.enableEdgeToEdge  
import androidx.compose.foundation.layout.fillMaxSize  
import androidx.compose.material3.Scaffold  
import androidx.compose.ui.Modifier  
import com.learning.countermvvm3.ui.theme.CounterMVVM3Theme
```

```

class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        enableEdgeToEdge()

        setContent {
            CounterMVVM3Theme {
                Scaffold(
                    modifier = Modifier.fillMaxSize()
                ) { innerPadding ->

                    CounterScreen(
                        modifier = Modifier.fillMaxSize(),
                        contentPadding = innerPadding
                    )
                }
            }
        }
    }
}

```

The generated theme name may be different.

Use the theme name created by Android Studio in your project.

Step 5: Add the following line in your GradleScripts/libs.version.toml file under the Libraries section:

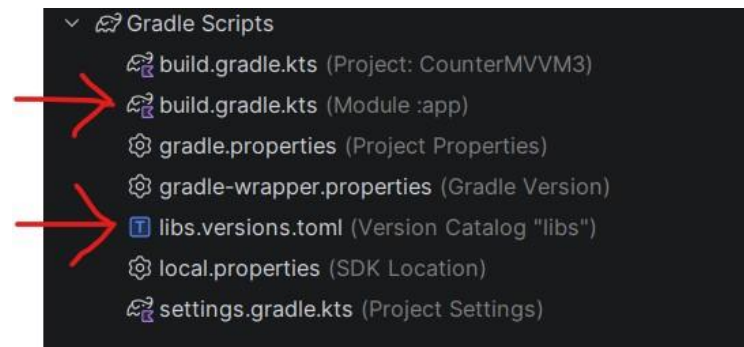
```

androidx-lifecycle-viewmodel-compose = { group = "androidx.lifecycle", name = "lifecycle-viewmodel-compose", version.ref = "lifecycleRuntimeKtx" }

```

Step 6: Add the following line in your GradleScripts/build.gradle.kts (:app) file under Dependencies:

`implementation(libs.androidx.lifecycle.viewmodel.compose)`



Identifying the MVVM Parts

Model

```
data class CounterState(  
    val count: Int = 0  
)
```

The Model stores the application data.

View

```
@Composable  
fun CounterScreen()
```

The View displays the counter and buttons.

ViewModel

```
class CounterViewModel : ViewModel()
```

The ViewModel stores the state and contains the counter logic.

Why Use MVVM?

MVVM provides several benefits.

Separation of Code

The UI and application logic are placed in separate classes.

Easier Maintenance

The screen design can be changed without changing the counter logic.

Easier Testing

The ViewModel functions can be tested separately.

Better Organisation

Each file has a specific purpose.

State Survives Rotation

A ViewModel can keep the counter value during common configuration changes such as screen rotation.

Important MVVM Rule

The View should display information and report user actions.

The ViewModel should manage the data and logic.

Therefore, avoid placing the main counter logic directly inside the button.

Avoid:

```
Button(  
    onClick = {  
        count++  
    }  
)
```

Prefer:

```
Button(  
    onClick = {  
        counterViewModel.increment()  
    } )
```

Demonstration

Follow these steps during the lesson:

1. Run the application.
2. Click Increase three times.
3. Confirm that the counter displays 3.
4. Click Decrease once.
5. Confirm that the counter displays 2.

6. Click Reset.
 7. Confirm that the counter displays 0.
 8. Click Decrease while the counter is zero.
 9. Confirm that the value does not become negative.
 10. Increase the value and rotate the emulator.
 11. Observe that the ViewModel retains the counter during rotation.
-

Student Exercise

Extend the application with the following features.

Task 1: Add Two

Create a button called:

Add 2

Create the following ViewModel function:

```
fun addTwo()
```

Task 2: Subtract Two

Create a button called:

Subtract 2

The counter must not become negative.

Task 3: Add a Message

Update the data class:

```
data class CounterState(  
    val count: Int = 0,  
    val message: String = "Counter is zero"  
)
```

Display:

Counter is zero

when the count is zero.

Display:

Counter is active

when the count is greater than zero.